Medium Analytics

Jesse Kim, Harsha Jagarlamudi, Jenna Yang, Ittoop Shinu Shibu

December 20th, 2019

I. Introduction and Purpose

Medium.com is an online article publishing platform for freelance writers and creators. Medium also hosts a social media aspect in conjunction with their articles and features a followers and likes (claps) system. The following system is put in place for users to read new articles posted by authors they enjoy reading and the claps system allows readers to show support for articles they read as well as potentially give the author a little bit of money via claps.

Because this publishing platform is highly text-based, natural language processing (NLP) is a very useful tool in analyzing trends and shifts within the thousands of articles on this site. NLP is at the intersection of computers and natural, human language found in these articles. It is important because it can help us determine more common occurrences of words or phrases alongside finding more unique ones that can supplement or best summarize an article.

The purpose of our project is to perform data analysis on the Medium platform as a whole. The project is open ended and gave us the freedom to explore or create anything regarding the website as long as it involved making use of the data. We want to learn about trends in the data that results in more profit for writers and how different words can classify a certain article to a topic.

II. Relevant Literature

While researching information that would be helpful for web scraping Medium.com, we found links and projects outlining similar tasks, albeit in a more general scope, such as a guide for web scraping news articles in Python (Zafra 1). The guide went through the general process of scraping an article using BeautifulSoup, which was a helpful reference, but still differed from our project as Medium.com had very unique divs and tags that we needed to consider separately. Moreover, we found a dataset for Medium data science articles on Kaggle, which was web scraped from Medium with a Python script (Hsankesara 1). Similar to our goal dataset, the Kaggle dataset included columns for claps, read time, author, text, and more. While the premise was very similar, our project was unique in that we wished to scrape more recent articles (from 2019 compared to 2017 in the Kaggle dataset) and our scraping would span four different topics (Tech, Sports, Money, Politics) while the Kaggle dataset exclusively contained only data science and ML/AL articles.

Part I. Web Scraping

In order to collect the data needed for analysis and modeling, data was web scraped from Medium.com. There were sources online which had Medium Article data, but most datasets were web scraped over a year ago,

didn't have all the relevant features, or were too small to build a classifier. Having articles in varying topics was crucial for our classifier and popular topics made for more useful classification. In addition, the more recent the data, the more representative the data would be for classifying future articles. Articles were scraped from the topics Politics, Technology, Money, and Sports. The same methods outlined below were used individually for all the different topics.

I. Methods

First, the robots.txt was checked to ensure proper access was granted before information was gathered. This is crucial to any web scraping process as scraping sensitive information may lead to potential legal issues. Once it was noted that web scraping articles was allowed, a webdriver extension Selenium was used to scroll down the infinite scrolling topic page. This took time as sending an abundance of GET requests in a short period of time would be bad practice in misuse of the server. Since Medium.com only keeps up relatively recent articles under its topic page (~2 years), once the page reached the end of its scrolling capacity the browser page source html was saved as a variable for further use. Regex was used to extract and standardize the htmls so they were usable. Each of the htmls led to an individual article written under that topic.

The list of htmls was looped through and a function applied to each which web scraped the article at the link. Once the 200 response was received from the GET request for the article html, the article data was retrieved through Beautiful Soup and regex. The page structure had to be explored to see the specific tags for certain features and try/except statements had to be used as certain pages lacked information other pages had (ie. certain pages didn't have any claps or had additional features). The resulting dictionary from the loop was converted into a pandas dataframe with 11 features: url, title, author, username, user_since, following, followers, published, claps, text, tags. The data was pushed to a csv for further analysis.

II. Data

Using the above method, four datasets were generated with the same 11 features (noted above). The Politics dataset had data on 3306 unique articles, after getting rid of duplicates and clap anomalies. The Sports dataset had 2588 articles, Technology has 4122 and Money had 1779 articles. The discrepancy was mainly due to certain topics having more articles written versus others had less and thus Selenium timed out faster while scrolling.

III. Exploration and Visuals

In order to analyze the data gathered from Medium.com, exploratory visualizations were developed to find patterns and trends.

When analyzing the tags, we wanted to answer the question: what were the most common tags that authors chose to use in each topic?
The following bar graphs (Fig. 1,2,3,4) reveal the top 10 tags in Tech, Sports, Money and Politics respectively.

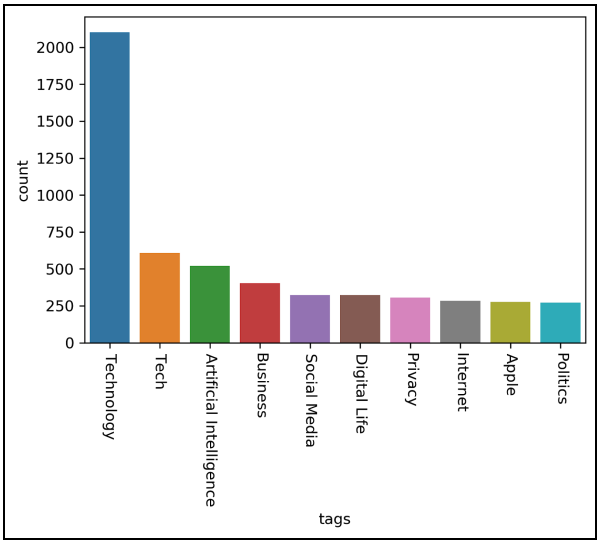

Figure 1: Most Frequent Tags in Technology

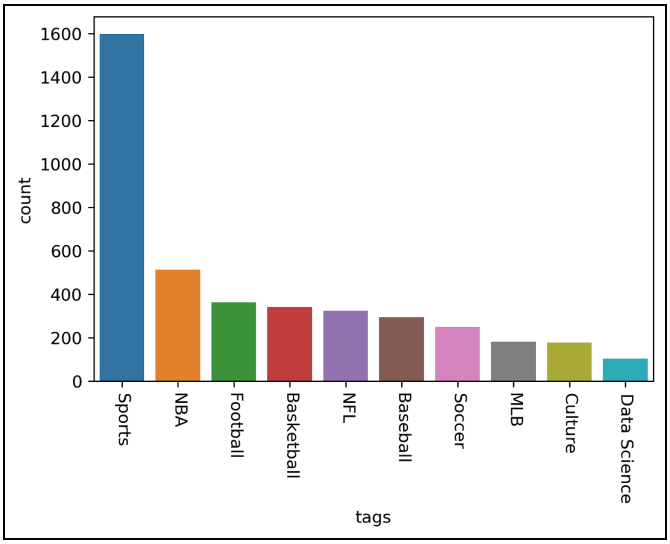

Figure 2: Most Frequent Tags in Sports

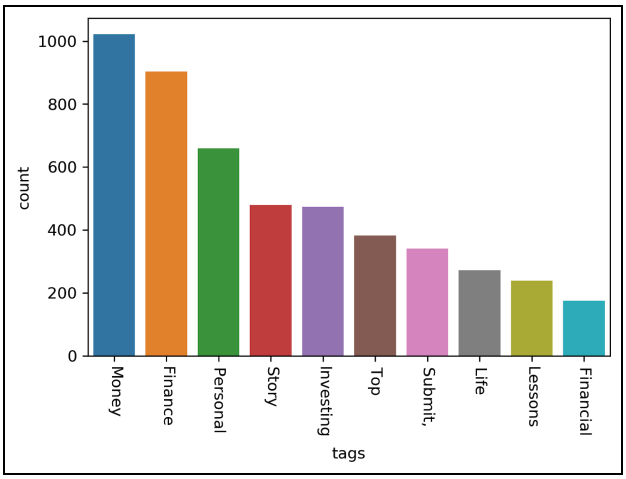

Figure 3: Most Frequent Tags in Money

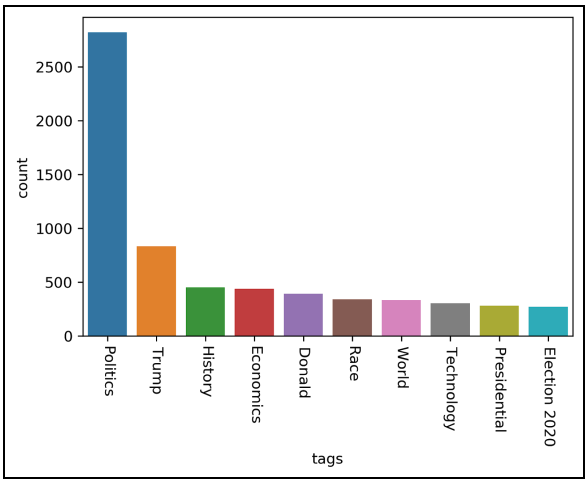

Figure 4: Most Frequent Tags in Politics

Given the visualizations, it was clear that among all the topics, the most frequent tag by far was the topic itself. Moreover, we found several intersecting tags such as politics in the Technology topic or data science in the Sports topic, exhibiting the multifaceted nature of Medium articles, which often cover many different, intersecting topics or domains.

Using the collected data, we also created visuals of how the data we found related to claps. The first set of visuals (Figures 5-7) shows a categorical scatter plot and a line graph of averages comparing claps to the month the article was published. The second set of visuals (Figures 8-10) is a density plot relating claps to the length of the article's title.

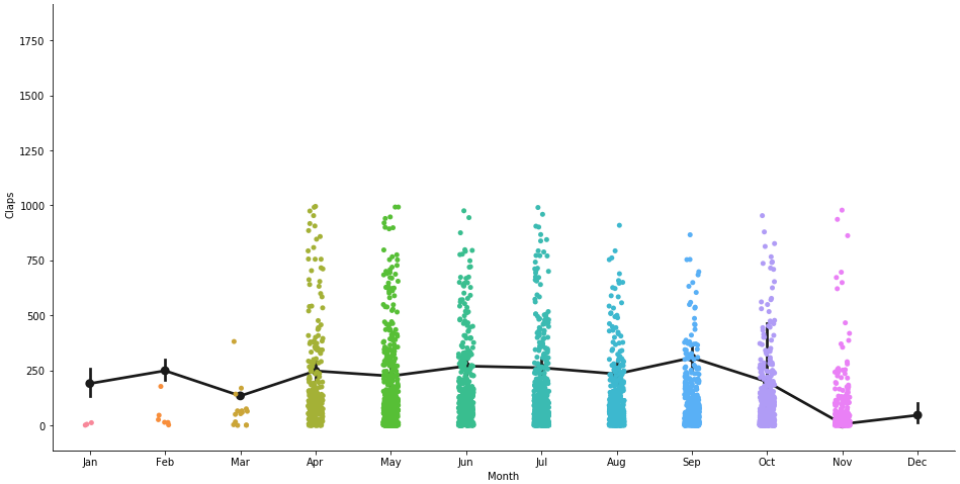

Figure 5: Month vs. Claps for Politics

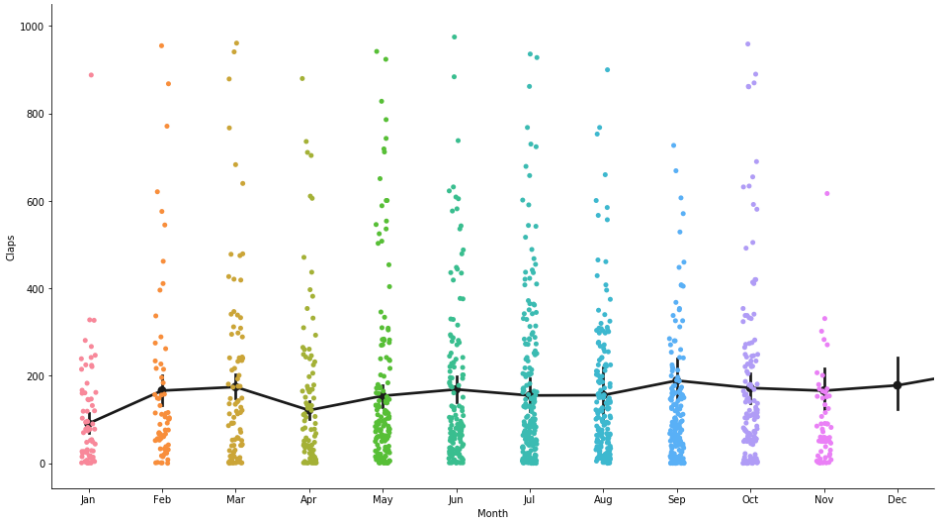

Figure 6: Month vs. Claps for Money

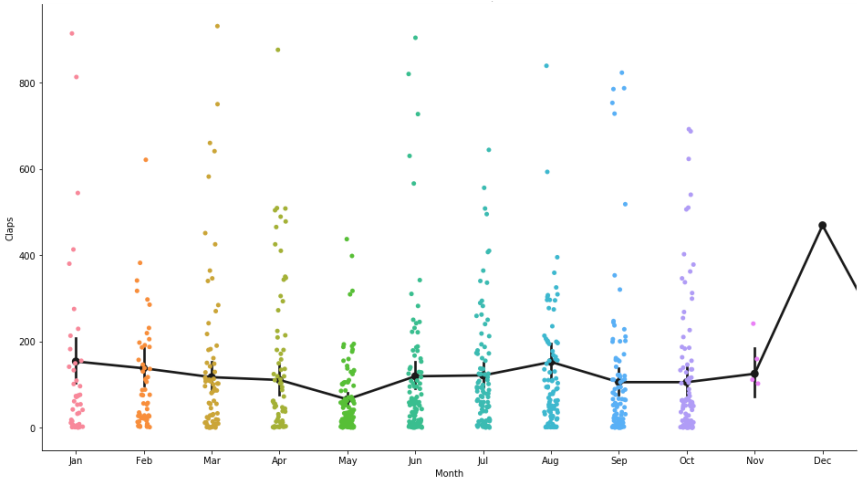

Figure 7: Month vs. Claps for Art

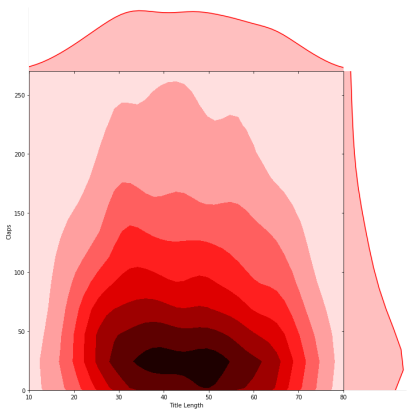

Fig. 8 Title Length vs. Claps (Politics)

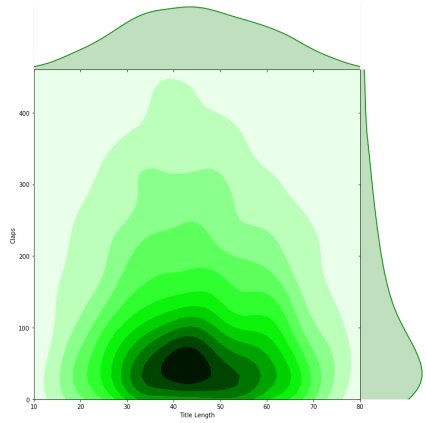

Fig. 9 Title Length vs. Claps (Money)

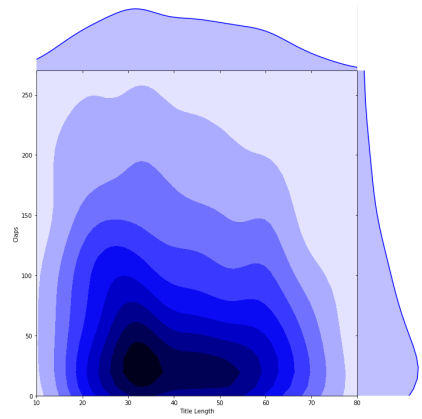

Fig. 10 Title Length vs. Claps (Art)

In the categorical months vs claps data, we found that each topic had their own popularity trends with coinciding seasons. Politics had more interest around election season, money had more interest around tax season, and art had more interest around the summer where vacations were common. In the density plot of title length vs claps, we found each topic to have their own ranges where both the title length and claps were consistent. Politics most commonly had between 35 and 55 characters in the title with claps between 20 and 30. Money most commonly had between 40 and 45 characters in the title with claps between 30 and 70. Art most commonly had between 30 and 35 characters in the title with claps between 20 and 40. These conclusions are important because authors can use these trends to write and ideal title in the proper time to maximize claps in their given category.

Next, varying topics had their titles segmented into words and the relative frequencies of words plotted. This helped compare what articles were most likely written about in varying topics.

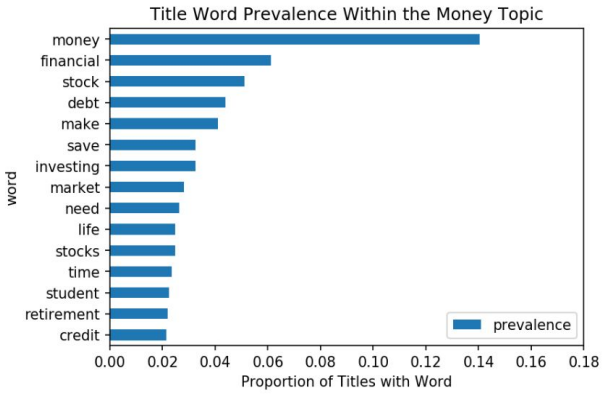

Fig. 11 Most Common Words in Money Articles Titles

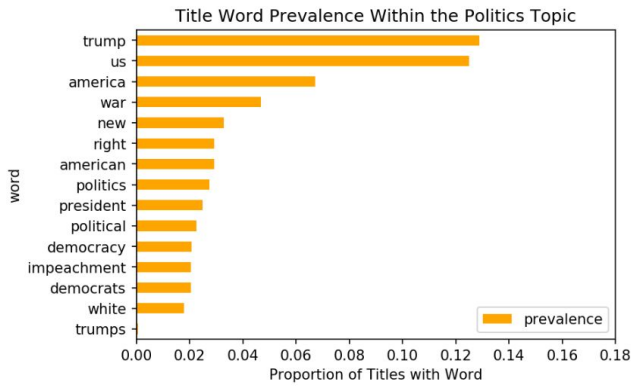

Fig. 12 Most Common Words in Politics Articles Title

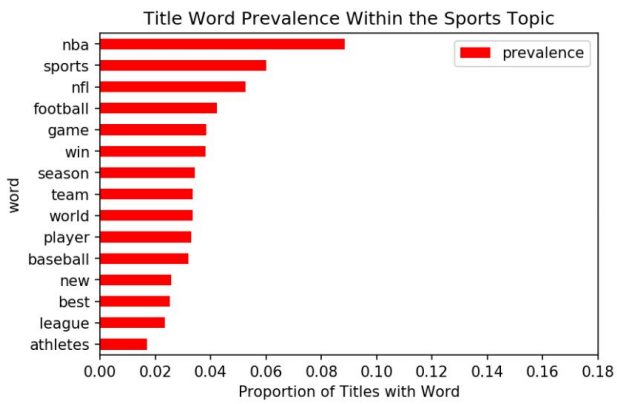

Fig. 13 Most Common Words in Sports Articles Titles

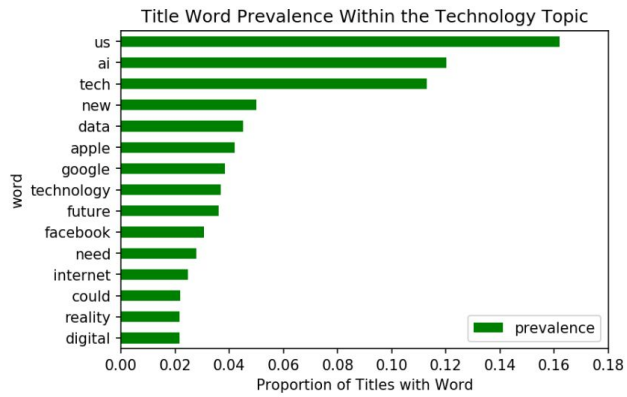

Fig. 14 Most Common Words in Tech Articles Title

The articles are from within the past few years, showcasing the most popular subjects within the topics (assuming the title is representative of the article). For the topic of Money, Figure 11 shows ‘Money’ is first, which is reasonable. There are quite a few terms about "stocks" (“investment”, “market”, “stock”, “stocks”), thus people may go to Medium.com Money section to see if there's a better way to invest/get insight into stock trends. Within the Politics topic, Figure 12 displays “Trump”, “US”, and “America” as the top three. It seems as though people have a considerable amount to say about the current president. The fact that "impeachment" is present highlights current events and more analysis could be done to track its usage now as the topic is extremely relevant. Perhaps the relative frequency of a word could lead to potential predictive power. For Sports, Figure 13 shows overall there aren't certain terms that totally dominate the title space like the other topics. “NBA” seems to be most prevalent, possibly due to the popularity of the sport. Then “nfl” shows up and then “baseball”, possibly denoting the order of popularity. Regarding technology, Figure 14 showcases that “US”, “AI”, and “Technology” are the top three. This shows the focus on AI and relative boom in articles on the subject. This may suggest that the US is leading or has a relatively large presence in the sector. (This could be due to Medium being big within the US rather than other countries as well).

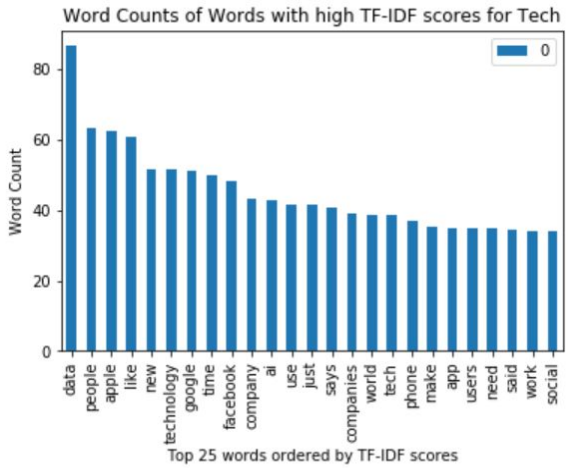

Fig. 15 Most Common Words in Tech Articles Text

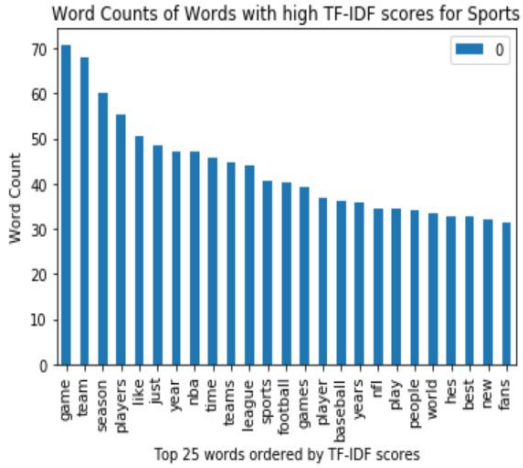

Fig. 16 Most Common Words in Sports Articles Text

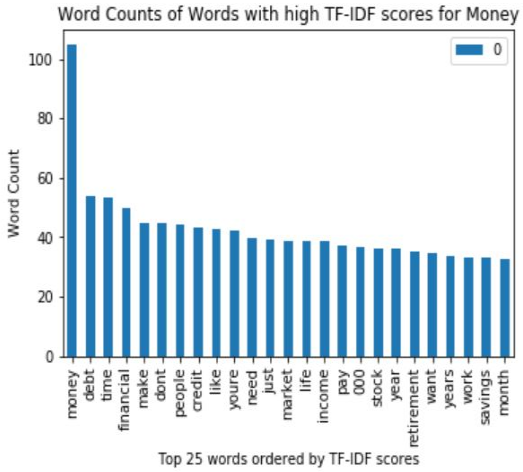

Fig. 17 Most Common Words in Money Articles Text

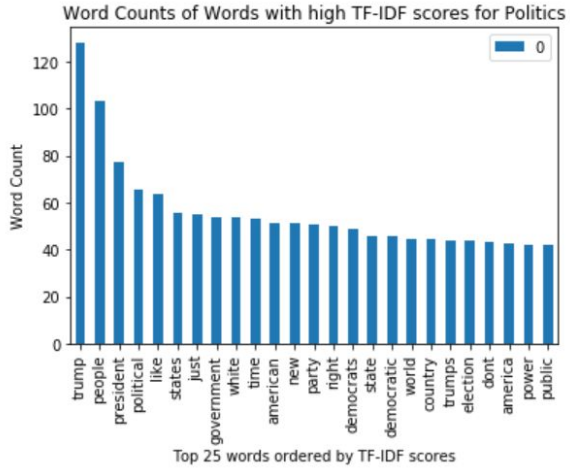

Fig. 18 Most Common Words in Politics Articles Text

A look into the most common words with the highest tf-idf scores from articles in each category shows similar results. For Technology (Figure 15), the top three most common words are “Data”, “People” and “Apple”. This makes sense as the Technology topic is very data and user centric and the company that drives a lot of technological change is Apple. For Sports (Figure 16), “Game”, “Team” and “Season” are the top three common words. These words are commonly associated with Sports hence, this is not a surprise. For the topic Money (Figure 17), the top three words are “Money”, “Debt” and “Time” which are words closely associated with Money. These words show what is currently weighing on people’s minds. The final topic, Politics (Figure 18), has “Trump”, “People” and “President” as the top three words. This maybe because of all the talk of Trump getting impeached. Hence, there are very distinct words that can define each category.

Part II : Modeling

I. Models

We used three different classifiers to classify our articles.

The first model we implemented to classify articles was decision trees. Decision tree classifiers work by repeatedly splitting the data by “decisions” (varying attribute values) until the data is in almost “pure” clusters (one topic) that can be grouped together to be considered one topic.

We also used k-nearest neighbors to classify our articles. K-nearest neighbors is a supervised machine learning algorithm often used for classification problems such as our problem of classifying Medium articles. KNN classifies a new input point by looking at the closest k points in terms of distance, and classifying based on the classification of those points. Essentially, KNN works under the assumption that similar things are closer together.

The last model we implemented was the multinomial naïve bayes model. The multinomial naïve bayes model is best known for its ability to classify text. It is based on conditional probability and a version of the naïve bayes, but while naïve bayes operates on the idea that features are independent of each other, multinomial naïve bayes fixes that issue. It does this by taking the multinomial distribution of each unique feature into account.
The prediction for the model is by first converting each of the features into distributions parameterised by vectors. These vectors are calculated by the following formula:

$$\hat{\theta}_{yi} = \frac{N_{yi} + \alpha}{N_y + \alpha n}$$

Where $N_{yi} = \sum_{x \in T} x_i$ represents the number of times a certain feature i appears in a sample class y in the training set T, and $N_y = \sum_{i=1}^{n} N_{yi}$ is the total number of features for a certain class y.
We used sklearn's Multinomial to implement the multinomial naïve bayes.

II. Preparation of the Data

To prepare our data for the models, we removed null and problematic values from the data to clean it up for training. Moreover, we randomly shuffled and sampled 2000 entries from our Tech and Politics tables while keeping around 1600 entries for Money and 2500 for Sports to try and keep the classes relatively balanced.

We needed to break our data into training and test sets. We added labels to our four tables and utilized an 80/20 split to divy our data into training and test sets. Lastly, we shuffled the sets to ensure randomness and obtained our X and Y data from both sets. For simplicity purposes, we preprocessed our text data using term frequency - inverse document frequency (tf-idf) vectorizers, that assigned values to words based on how unique or overused they were. Our vectors focused on the top 50 max features for decision tree and k nearest neighbors, and we used this to ultimately fit our data into a transformed version. We used all the tf-idf features for our multinomial naive bayes model for better classification of our data.

III. Optimization

To optimize the Decision Tree model for the final testing, we performed cross validation alongside hyperparameter tuning to get ideal values for both the min_sample_splits and max_depth parameters. The min_sample_splits parameter defines the minimum proportion of the data that needs to be in each side of the split of data and the max_depth parameter defines how many splits the tree can go up to. Cross validation split up our train set into three sections which were used as 2 train and 1 test set and swapped around to find which parameters were best.

Unlike other algorithms, KNN did not require much hyperparameter tuning- the only parameter we needed to the tune was *k*, the number of neighbors. To do this, we performed 10-fold cross validation and plotted our accuracies for different k's, then found our specific k value that optimized our cross validation score, which was 23.

For the Multinomial Naive Bayes model, we tuned the model by playing around with the n grams each model used as features. We created pipelines to implement passing in ngrams of our choosing and tested out the validation dataset to see if the model was being improved. After testing various ngrams, we came to the conclusion that the ngram with the highest validation score combined was unigrams.

IV. Results and Analysis

The resulting Decision Tree model had a final accuracy of about 83% and had only a notable issue discerning sports from tech, as seen in the confusion matrix below.

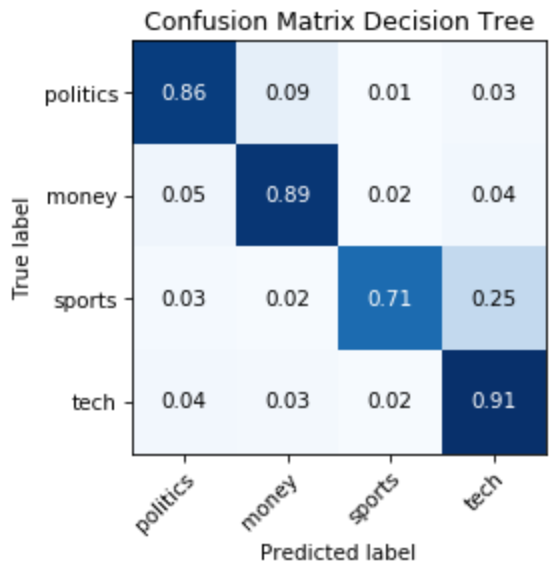

Figure 19: Decision Tree Confusion Matrix

Similarly, the result KNN model had a final accuracy of about 88% with the confusion matrix shown below.

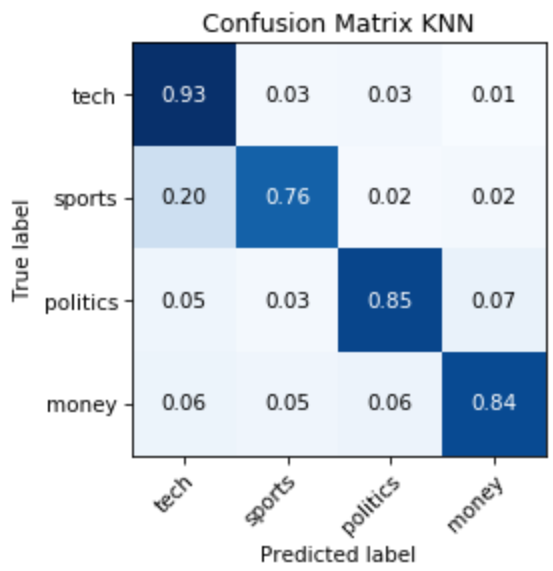

Figure 20: K-Nearest Neighbors Confusion Matrix

The Multinomial Naive Bayes had the highest accuracy of 93% and did not have a lot of trouble classifying most of the articles in the right category as shown by the confusion matrix below.

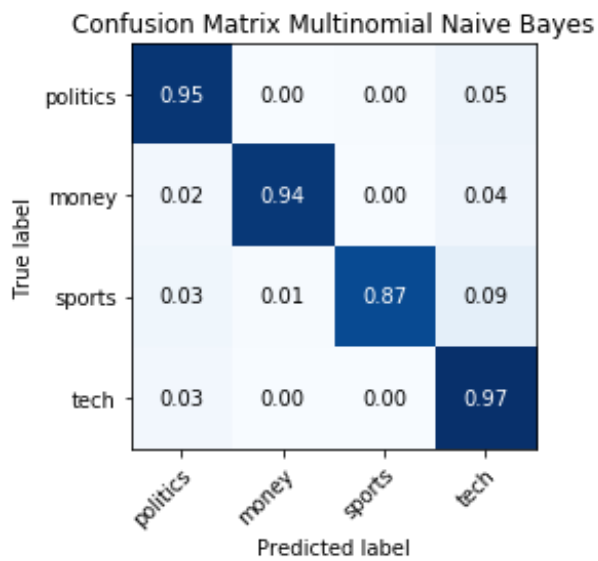

Figure 21: Multinomial Naive Bayes Confusion Matrix

V. Part II Overall Conclusion

Overall, our models performed well in classifying the topics; however, there was most error when it came to Tech and Sports, where the classifier often confused the two topics. As shown with our tags visualization for the Sports topic, we hypothesize that this confusion is due to the fact that many of the sports and tech articles

on Medium overlap and intertwine. This is likely because that given the academic and educational nature of Medium, many sports articles deal with data analytics, or statistical projects.

When comparing the three models, the decision tree has the lowest accuracy, the k nearest neighbors has the second highest accuracy and the multinomial naive bayes has the highest accuracy. The decision tree might have the lowest accuracy because it is difficult to create decision boundaries if the articles contain words that are shared between various categories. This is further proven by the confusion matrix (Figure 19) which shows that there were a lot of mislabeled articles between sports and technology. This means that the model found it hard to find proper decision boundaries for the articles from different categories.

The k nearest neighbor also faces the same issue. Since the classifier classifies by getting the distance between features of each article, an article that contains words that probably fall under more than two categories would be hard to classify. This is reflected in the confusion matrix(Figure 20) which shows that the model misclassified articles between sports and technology.

Finally the last model, multinomial naive bayes, has the highest accuracy compared to the other models. This may be because this model used the most text features compared to the other models; the other two models used the top 50 features while the multinomial naive bayes used all the text features. Compared to the other models, the multinomial naive bayes confusion matrix(Figure 21) had the least misclassified labels. This might be because the probabilistic model hence, it classifies articles that have certain words that are more common in specific categories more easily than the other two models.

In conclusion, all the models that we used to classify topics were very good. However, the multinomial naive bayes ended up getting the best accuracy because it depended on the probability of each feature. Hence, words that were more likely to end up in specific topics would make classification easy.

VI. Future Explorations

Moving forward, creating a keyword generator to determine possible keywords for articles written would be beneficial. A maximum of five keywords can be used for each article, and determining the right keywords may be crucial for articles to be seen and read. The right balance of keyword popularity with keyword relevance would serve to improve the chances of the article being seen and enjoyed by the correct audience. Another possible direction the team could head is to create a model which predicts the number of claps an article could get based on the features of the article. This is definitely more of a challenge as it would require a regression model on heavily skewed data, but could potentially be interesting as having a large number of claps can generate money for the author. Thus, determining what makes an article successful could be highly beneficial.

VII. References

[1] Zafra, Miguel Fernandez. “Web Scraping News Articles in Python.” Medium, Towards Data Science, 9 July 2019, https://towardsdatascience.com/web-scraping-news-articles-in-python-9dd605799558.
[2] Hsankesara. “Medium Articles.” Kaggle, 17 June 2018, https://www.kaggle.com/hsankesara/medium-articles/kernels.

[3] “Get Smarter about What Matters to You.” Medium, https://medium.com/.
[4] “Learn.” Scikit, https://scikit-learn.org/stable/.
[5] “Natural Language Toolkit.” Natural Language Toolkit - NLTK 3.4.5 Documentation, https://www.nltk.org/.
[6] Nelson, Dan. “Overview of Classification Methods in Python with Scikit-Learn.” Stack Abuse, Stack Abuse, 11 May 2019, https://stackabuse.com/overview-of-classification-methods-in-python-with-scikit-learn/.
[7] AlindGuptaCheck. “Elbow Method for Optimal Value of k in KMeans.” GeeksforGeeks, 6 June 2019, https://www.geeksforgeeks.org/elbow-method-for-optimal-value-of-k-in-kmeans/.
[8] Opidi, Alfrick. “A Gentle Introduction to Text Summarization in Machine Learning.” FloydHub Blog, FloydHub Blog, 17 Sept. 2019, https://blog.floydhub.com/gentle-introduction-to-text-summarization-in-machine-learning/.
[9] “Implementing Web Scraping in Python with BeautifulSoup.” GeeksforGeeks, 18 Nov. 2016, https://www.geeksforgeeks.org/implementing-web-scraping-python-beautiful-soup/.
[10] “Robots.txt File: 2019 SEO Best Practices.” Moz, https://moz.com/learn/seo/robotstxt.
[11] “Sklearn.naive_bayes.MultinomialNB¶.” Scikit, https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html.
[12] Garbade, Michael J. “A Simple Introduction to Natural Language Processing.” Medium, Becoming Human: Artificial Intelligence Magazine, 15 Oct. 2018, https://becominghuman.ai/a-simple-introduction-to-natural-language-processing-ea66a1747b32.
[13] “Statistical Data Visualization¶.” Seaborn, https://seaborn.pydata.org/.
[14] “Sklearn.tree.DecisionTreeClassifier¶.” Scikit, http://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html.
[15] Chakure, Afroz. “Decision Tree Classification.” Medium, Towards Data Science, 7 July 2019, https://towardsdatascience.com/decision-tree-classification-de64fc4d5aac.
[16] Harrison, Onel. “Machine Learning Basics with the K-Nearest Neighbors Algorithm.” Medium, Towards Data Science, 14 July 2019, https://towardsdatascience.com/machine-learning-basics-with-the-k-nearest-neighbors-algorithm-6a6e71d01761.
[17] Sun, Qiping. “How to Deal with Cross-Validation Based on KNN Algorithm, Compute AUC Based on Naive Bayes...” Medium, Medium, 18 May 2018, https://medium.com/@svanillasun/how-to-deal-with-cross-validation-based-on-knn-algorithm-compute-auc-based-on-naive-bayes-ff4b8284cff4.